Bot de Gestión de Obra — *Aestimus Final*

Automatización de registro de avance con Telegram, IA y Google Sheets

---

¿Qué hace este sistema?

Este flujo de automatización actúa como un **asistente inteligente de campo** para obras de construcción. Permite que cualquier miembro del equipo envíe información de avance de obra directamente desde **Telegram** — ya sea escribiendo un mensaje de texto, adjuntando una fotografía o cargando un archivo `.txt` con múltiples registros — y que esa información quede **automáticamente organizada y guardada** en una hoja de cálculo de **Google Sheets**, lista para ser utilizada en la generación de reportes, presupuestos y paneles fotográficos.

```
📱 Usuario en Telegram
        ↓
🤖 Bot Aestimus (n8n)
   ├─ Archivo .txt → 📄 Procesamiento directo por código
   └─ Mensaje/foto → 🧠 Clasificador IA (Groq)
        ↓
📊 Google Sheets "Raw data"
```

Tipos de entrada soportados (4 clases)

El sistema reconoce **cuatro formatos de entrada** válidos, diseñados para cubrir tanto el seguimiento BIM como el seguimiento tradicional por partidas:

Clase	Descripción	¿Lleva foto?	¿Cómo se envía?	¿Quién la procesa?
1A	Datos BIM de **un elemento**	☑ Opcional	Mensaje de texto	Clasificador IA
1B	Datos BIM de **múltiples elementos**	❌ No	Archivo `.txt`	Código JavaScript
2A	Datos Tradicionales de **una partida**	☑ Opcional	Mensaje de texto	Clasificador IA
2B	Datos Tradicionales de **múltiples partidas**	❌ No	Archivo `.txt`	Código JavaScript

> **Sinónimos aceptados:** el bot entiende "Avance" como "% de Ejecución", "ID" como "Id Elemento", "Código" como "Código de Partida", "Sub" o "Subpartida" como "Subcódigo", "Fecha ejecutada" como "Fecha de Ejecución".

> **Fechas flexibles:** se aceptan separadores con guiones (`25-03-2025`), slashes (`25/03/2025`) o puntos (`25.03.2025`). El sistema normaliza siempre a formato `YYYY-MM-DD`.

> **Números flexibles:** el `%` de avance y las unidades del metrado son opcionales en el mensaje. El sistema extrae el número puro.

Archivos .txt: el formato de carga masiva

Para las clases **1B** y **2B**, el sistema acepta archivos de texto plano (`.txt`) con estructura tabular.

Separadores soportados: tabulación (`Tab`) o coma (`,`) — detectados automáticamente.

Codificaciones soportadas: `Latin-1` y `UTF-16` — detectadas automáticamente.

Con encabezados (recomendado)

El sistema reconoce los encabezados de forma flexible (sin distinguir tildes ni mayúsculas):

Clase 1B — 3 columnas	Clase 2B — 5 columnas
`Id Elemento`	`Código de Partida`
`Fecha de Ejecución`	`Subcódigo`
`% de Ejecución`	`Metrado Ejecutado`
	`Fecha de Ejecución`
	`% de Ejecución`

Sin encabezados

Si el archivo no tiene fila de encabezado, el sistema determina la clase **por el número de columnas**:

- **3 columnas** → Clase 1B → orden esperado: `Id Elemento | Fecha | % Avance`
- **5 columnas** → Clase 2B → orden esperado: `Código | Subcódigo | Metrado | Fecha | % Avance`

Descripción de nodos del flujo

🟤 Entrada y detección

1. Telegram Trigger Punto de entrada del flujo. Escucha todos los mensajes entrantes al bot. Descarga automáticamente los archivos adjuntos (imágenes y documentos).

2. ¿Tiene archivo .txt? *(Nodo IF)* Primera y más importante bifurcación del flujo. Revisa si el mensaje incluye un documento con extensión `.txt`.

- **Sí** → **rama de archivos** (procesamiento por código, sin IA).
- **No** → **rama de mensajes** (clasificación por IA).

> Esta separación es clave: los archivos `.txt` **nunca pasan por el Clasificador IA**. Son procesados íntegramente por código JavaScript, lo que los hace más rápidos, deterministas y sin consumo de tokens.

Rama de archivos .txt (Clases 1B y 2B)

3. Leer contenido .txt *(Telegram — Get File)* Descarga el contenido binario del archivo `.txt` desde los servidores de Telegram.

4. Procesar contenido *(Nodo Code — JavaScript)* Núcleo del procesamiento masivo. Opera en dos estrategias:

- **Estrategia A — Con encabezados:** detecta el separador (Tab o coma), normaliza los encabezados (sin tildes, minúsculas) y mapea las columnas por términos clave. Determina la clase según si encuentra `Id Elemento` (→ 1B) o `Código de Partida` (→ 2B).
- **Estrategia B — Sin encabezados:** si la primera fila no contiene términos reconocibles, cuenta el número de columnas: 3 columnas → Clase 1B, 5 columnas → Clase 2B, asignando los campos por posición fija.

En ambos casos produce un array de filas estructuradas listas para guardar.

Rama de mensajes de texto/imagen (Clases 1A y 2A)

5. Clasificador IA *(LLM Chain — Groq · `llama-3.1-8b-instant`)* Cerebro del flujo para mensajes de texto. Recibe únicamente el texto o caption del mensaje y extrae:

- La clase (`1A` o `2A`), guiándose también por la cantidad de datos separados por comas: **3 datos = 1A**, **5 datos = 2A**.
- Todos los campos requeridos en JSON estricto.
- Manejo de mensajes compactos (sin espacios, solo comas como separadores).

El prompt está deliberadamente **simplificado a solo 1A, 2A e INVÁLIDO**, ya que las clases 1B y 2B son interceptadas antes por la rama `.txt` y nunca llegan a este nodo. Esto reduce tokens, mejora la velocidad y elimina confusión en el modelo.

6. Text a json *(Nodo Code — JavaScript)* Convierte la respuesta de texto del modelo IA en un objeto JSON utilizable. Limpia posibles marcadores de markdown que el modelo pudiera incluir.

Además ejecuta la **detección de fotografía por metadatos de Telegram**: en lugar de confiar en el modelo (que no puede ver imágenes), lee directamente el campo `message.photo` del mensaje original —usando `.first()` para garantizar referencia al mensaje correcto— y sobreescribe el campo `tiene_foto` con el resultado real.

Validación y enrutamiento

7. ¿Es inválido? *(Nodo IF)* Comprueba si la clase resultante (de cualquiera de las dos ramas) es `INVALIDO`.

- **Sí → Enviar error al usuario.**
- **No → Enrutar por clase.**

8. Enviar error *(Telegram — Send Message)* Si el mensaje no es clasificable o el archivo `.txt` tiene un formato irreconocible, el bot responde con un **mensaje formateado** que incluye:

- El motivo específico del error.
- La guía completa de los 4 formatos válidos con un mini-ejemplo de cada uno.

```
❌ [motivo del error]

📋 Formatos de entrada válidos:

◇ 1A — Elemento BIM por mensaje (± foto)
  Ej: E-101, 25-03-2025, 75

◇ 1B — Múltiples elementos BIM (.txt)
  Ej fila: E-101, 25-03-2025, 75

◇ 2A — Partida tradicional por mensaje (± foto)
  Ej: 03.01.02, 3, 12.5, 25-03-2025, 60

◇ 2B — Múltiples partidas (.txt)
  Ej fila: 03.01.02, 3, 12.5, 25-03-2025, 60
```

9. Enrutar por Clase *(Nodo Switch)* Distribuye el flujo según la clase detectada hacia el camino de guardado correspondiente:

- **1A / 2A** → Verificar si tiene imagen.
- **1B / 2B** → Preparar filas de TXT para guardado en bucle.

🟢 Preparación y guardado de datos

10. ¿Se tiene una imagen? *(Nodo IF)* Para las clases 1A y 2A, verifica si hay fotografía adjunta.

- **Sí → Obtener y subir imagen a Drive.**
- **No → Preparar fila directamente sin foto.**

11. Obtener Imagen *(Telegram — Get File)* Descarga la imagen de Telegram en su resolución más alta disponible.

12. Subir y renombrar imagen *(Google Drive)* Sube la imagen a una carpeta específica de Google Drive. La renombra automáticamente combinando los campos del registro (Id/Código, subcódigo, metrado, fecha, avance) para garantizar trazabilidad y evitar duplicados de nombre.

13. Preparar fila individual *(Nodo Code)* Para mensajes individuales (1A/2A): combina los datos del registro con la URL pública de la imagen en Google Drive (o `"No se registra"` si no hay foto), generando un objeto plano listo para Google Sheets.

14. Preparar filas TXT *(Nodo Code)* Para archivos `.txt` (1B/2B): mapea cada elemento del array de filas procesadas en un objeto individual con `url_foto: "No se registra"`, preparando todos los registros para el guardado en bucle.

15. Loop Over Items *(Split in Batches)* Itera registro por registro sobre el array de filas. Imprescindible para archivos `.txt` donde pueden llegar decenas o cientos de registros en una sola carga. Cada iteración guarda

una fila antes de continuar con la siguiente.

16. Guardar en Base Data *(Google Sheets — Append)* Inserta la fila al final de la hoja **"Raw data"** de Google Sheets. Columnas guardadas:

`Id Elemento | Código de Partida | Subcódigo | Metrado Ejecutado | Fecha de Ejecución | % de Ejecución | Fotografía`

17. Confirmación *(Telegram — Send Message)* Una vez completado el guardado, el bot notifica al usuario:

- Para archivos `.txt`: "✅ *¡Listo! El archivo fue procesado y todas sus filas fueron guardadas.*"
- Para registros individuales: "✅ *¡Listo! El elemento fue guardado exitosamente.*"

Modelo de IA y decisiones de arquitectura

Modelo del Clasificador: `llama-3.1-8b-instant` (Groq)

Elegido por su **velocidad extrema** y acceso gratuito. Para la tarea de clasificar mensajes de texto y extraer campos en JSON, un modelo de 8B parámetros es suficiente y eficiente.

⚠ Limitantes observadas

> [!WARNING] Al ser un modelo **solo de texto** (no multimodal), tiene restricciones concretas:

- **No puede ver imágenes.** Si el usuario envía una foto con caption, el modelo solo recibe el texto del caption. La detección de si hay foto adjunta fue trasladada al nodo de código (`Text a json`), que lee los metadatos de Telegram directamente — solución más fiable que depender del modelo.
- **Puede fallar con mensajes muy ambiguos o abreviados.** El prompt incluye la pista del conteo de comas (3 = 1A, 5 = 2A) para minimizar este riesgo.
- **Puede incluir formato inesperado en la respuesta** (por ejemplo, envolver el JSON en bloques de markdown). El nodo `Text a json` limpia esto antes de parsear.

Decisión de arquitectura: múltiples modelos especializados

El sistema utiliza **modelos diferentes para tareas diferentes**, distribuyendo la carga entre pools de rate limits independientes de Groq:

Tarea	Modelo	Razón
Clasificar mensajes de texto	`llama-3.1-8b-instant`	Velocidad máxima, tarea simple
Describir fotos en paneles	`llama-4-scout-17b` (Apps Script)	Visión nativa, tarea multimodal
Memoria descriptiva *(futuro)*	`llama-3.3-70b-versatile`	Redacción técnica larga y estructurada

Esta separación evita agotar el cupo de un solo modelo y asigna la herramienta correcta a cada trabajo.

💡 Modelos alternativos para el clasificador

Modelo	Proveedor	Ventaja
`llama-3.3-70b-versatile`	Groq	Mayor precisión en extracción JSON con textos ambiguos
`gemini-2.0-flash`	Google AI	Multimodal nativo — podría detectar imágenes directamente
`gpt-4o-mini`	OpenAI	JSON mode nativo, soporte multimodal, muy preciso

Sugerencias de mejora

> [!NOTE] **Validación de duplicados antes de guardar**
>
> El flujo actualmente no verifica si un registro idéntico ya existe en Sheets antes de insertar. Un usuario que envíe el mismo mensaje dos veces por error generaría filas duplicadas. Se podría añadir un nodo de búsqueda previo al guardado que consulte por `Id Elemento` + `Fecha` (o `Código de Partida` + `Fecha`) y rechace el registro si ya existe, notificando al usuario.

Conclusiones

El flujo **Bot-gestion-aestimus-final** es una solución práctica, eficiente y de bajo costo para digitalizar el registro de avance de obra en campo:

- ☑ **Accesibilidad total:** cualquier miembro del equipo registra desde Telegram, sin sistemas adicionales.
- ☑ **Doble vía de entrada inteligente:** la bifurcación temprana entre archivos `.txt` y mensajes de texto permite que cada tipo de dato sea procesado por el mecanismo más adecuado — código determinista para cargas masivas, IA para mensajes flexibles.
- ☑ **Tolerancia de formato:** acepta fechas con distintos separadores, números sin símbolo de unidad, sinónimos de campo y archivos `.txt` con o sin encabezados, con tabulaciones o comas.
- ☑ **Trazabilidad fotográfica:** imágenes subidas a Drive con nombres estructurados y URL vinculada al registro en Sheets, base para la generación futura de paneles fotográficos y memorias descriptivas.
- ☑ **Retroalimentación inmediata:** el bot siempre responde — confirmando el guardado o explicando el error con el formato correcto de referencia.
- ☑ **Arquitectura escalable:** diseñado para crecer. Los datos normalizados en Sheets son el punto de partida para la siguiente etapa: generación automática de paneles fotográficos, memorias descriptivas valorizadas y otros documentos técnicos de obra.